

5. Klasifikasi Fungsi Berdasarkan Laju Pertumbuhannya

Didepan kita telah melakukan penghitungan banyaknya operasi dasar yang dilakukan oleh suatu algoritma. Penghitungan yang dilakukan tersebut masih belum teliti, karena ada operator sampingan tidak dihitung. Sehingga hasil analisis kita masih kasar. Jumlah keseluruhan operasi yang dilakukan oleh suatu algoritma sebanding dengan banyaknya operasi dasar yang dihitung. Sebagai contoh, algoritma *SequentialSearch* melakukan kerja sebanyak $W(n) = n$, jumlah keseluruhan operasi yang dikerjakan oleh *SequentialSearch* adalah cn dengan c adalah konstanta positif.

Pada saat kita membandingkan dua algoritma, katakan algoritma A dan algoritma B. Misal Algoritma A mempunyai operasi dasar $W_1(n) = 2n$ atau jumlah operasi totalnya adalah $2c_1n$ dan algoritma B mempunyai operasi dasar $W_2(n) = 6n$ atau jumlah operasi totalnya adalah $6c_2n$. Pertanyaan yang muncul adalah: bagaimana ketelitian hasil perbandingan kedua algoritma tersebut ?. Atau pertanyaan yang lebih spesifik: mana yang lebih cepat ?. Jelas kita tidak mengetahui jawabannya. Jika konstanta $c_1 > 3c_2$ (misal algoritma A banyak operasi tambahan) maka algoritma A lebih lambat dari algoritma B. Atau bisa sebaliknya. Dua fungsi yang hanya berbeda pada faktor pengali konstanta, kedua fungsi tersebut berada dalam satu kelas. Sehingga algoritmanya mempunyai kompleksitas yang sama.

Sekarang kita ambil pemisalan yang lain. Anggap algoritma A mempunyai $W_1(n) = n^3$ atau jumlah operasi totalnya adalah c_1n^3 dan algoritma B mempunyai operasi dasar $W_2(n) = 5n^2$ atau jumlah operasi totalnya adalah $5c_2n^2$. Mana yang lebih cepat ?. Untuk nilai yang kecil $n < \frac{5c_2}{c_1}$ maka algoritma A akan lebih cepat dari pada algoritma B. Namun apabila $n > \frac{5c_2}{c_1}$, maka algoritma B lebih cepat dari pada algoritma A. Kalau kita lihat pada laju pertumbuhan fungsinya, fungsi kubik mempunyai laju pertumbuhan lebih besar dari pada fungsi kuadratik. Sehingga untuk ukuran input yang sangat besar, algoritma B jauh lebih baik dari pada algoritma A.

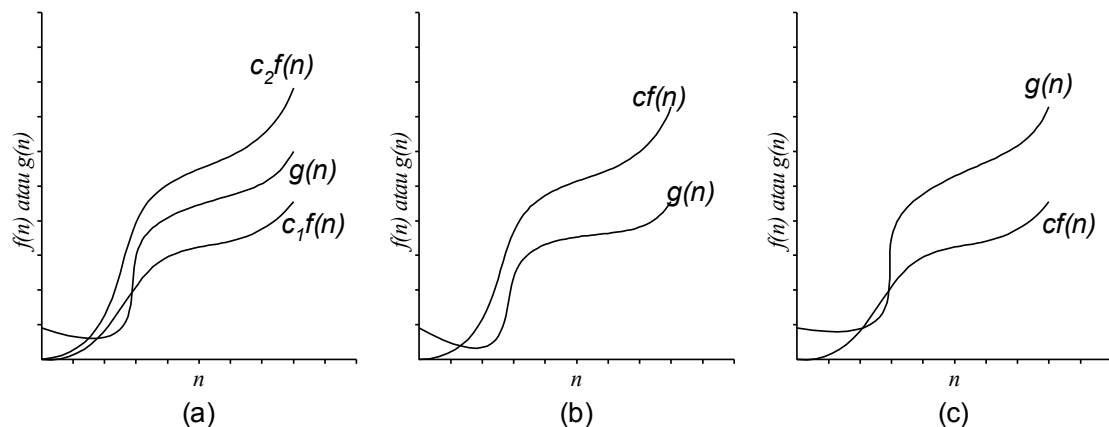
Dengan gambaran contoh di atas, kita ingin mempunyai suatu cara untuk mengklasifikasikan fungsi yang tidak tergantung pada konstanta dan mengabaikan nilai variabel bebas (ukuran input) yang kecil. Klasifikasi fungsi seperti ini berdasarkan **laju pertumbuhan asimptotik** atau **order** dari fungsinya.

Berikut ini akan ditampilkan beberapa definisi dan notasi yang umum dipakai. Misal N merupakan himpunan bilangan alam, R merupakan bilangan riil, R^+ merupakan bilangan riil positif, R^* merupakan bilangan riil non negatif, $f: N \rightarrow R^+$ dan $g: N \rightarrow R^+$ didefinisikan:

- $O(f(n)) = \{ g(n) \mid \text{untuk beberapa konstanta } c \in R^+ \text{ dan } n_0 \in N \text{ berlaku } g(n) \leq cf(n) \text{ untuk semua } n \geq n_0 \}$. Atau fungsi $g(n)$ didominasi secara asyptotik oleh fungsi $f(n)$.
- $\Omega(f(n)) = \{ g(n) \mid \text{untuk beberapa konstanta } c \in R^+ \text{ dan } n_0 \in N \text{ berlaku } g(n) \geq cf(n) \text{ untuk semua } n \geq n_0 \}$. Atau fungsi $g(n)$ mendominasi secara asyptotik fungsi $f(n)$.
- $\theta(f(n)) = \{ g(n) \mid \text{untuk beberapa konstanta } c_1, c_2 \in R^+ \text{ dan } n_0 \in N \text{ berlaku } c_1f(n) \leq g(n) \leq c_2f(n) \text{ untuk semua } n \geq n_0 \}$. Atau fungsi $g(n)$ mendominasi secara asyptotik fungsi $f(n)$ dan fungsi $g(n)$ didominasi secara asyptotik oleh fungsi $f(n)$.

Himpunan fungsi $O(f(n))$ sering disebut dengan '**big oh of $f(n)$** ' atau '**oh $f(n)$** '. Meskipun $O(f(n))$ merupakan himpunan, lebih paktis mengatakan ' $g(n)$ adalah oh $f(n)$ ' dari pada mengatakan ' $g(n)$ adalah anggota dari oh $f(n)$ '. Dan penulisan $g(n) \in O(f(n))$ lebih praktis ditulis dengan penulisan $g(n) = O(f(n))$. Himpunan fungsi $O(f(n))$ sering disebut dengan **Order $f(n)$** .

Kalau melihat definisi O dan Ω , ini merupakan dua definisi yang mirip karena pada dasarnya hanya masalah 'mendominasi'(kata kerja aktif) dan 'didominasi'(kata kerja pasif). Oleh karena itu kita lebih sering menggunakan salah satu saja yaitu O .



Gambar 1.7

- (a) Fungsi $f(n)$ mendominasi $g(n)$ dan $f(n)$ didomonasi oleh $g(n)$, (b) Fungsi $f(n)$ mendominasi $g(n)$, (c) Fungsi $f(n)$ didominasi $g(n)$.

Contoh 1:

Pandang fungsi $f(n) = \frac{1}{n} + 7$ dan $g(n) = n + 1$.

$f(n)$ didominasi oleh $g(n)$ atau $f(n) = O(g(n))$, karena: kalau diambil $c=1$ berlaku $f(n) \leq g(n)$ untuk semua nilai $n \geq n_0$ ($n_0 = 7$).

$$\frac{1}{n} + 7 \leq c(n + 1)$$

$$1 + 7n \leq c(n^2 + n)$$

$$1 \leq c(n^2 + n) - 7n$$

$$1 \leq n^2 - 6n \text{ untuk } c = 1 \text{ dipenuhi untuk semua nilai } n \geq 7.$$

Akan tetapi $f(n)$ tidak mendominasi $g(n)$ atau $f(n) \geq cg(n)$, karena: kalau dianggap $f(n)$ mendominasi $g(n)$ atau $g(n) \leq cf(n)$ untuk $n \geq n_0$, maka:

$$n + 1 \leq c\left(\frac{1}{n} + 7\right) \text{ untuk } n \geq n_0$$

$$n^2 + n \leq c(1 + 7n) \text{ untuk } n \geq n_0$$

Untuk $n = 7c$ didapat $49c^2 + 7c \leq c + 49c^2$. Hal ini tidak mungkin, oleh karena itu $f(n)$ tidak mendominasi $g(n)$.

Ada cara lain untuk menunjukkan bahwa $g(n) = O(f(n))$, yaitu:

$g(n) = O(f(n))$ jika

$$\lim_{n \rightarrow \infty} \frac{g(n)}{f(n)} = K \text{ untuk } K \in R^+$$

Ini mempunyai arti bahwa: jika nilai limit diatas ada (bukan tak berhingga), maka laju pertumbuhan fungsi $g(n)$ tidak lebih cepat dari laju $f(n)$. Dan jika nilai limitnya tak berhingga maka laju pertumbuhan fungsi $g(n)$ lebih cepat dari laju $f(n)$.

Contoh 2:

Pandang fungsi $f(n) = n^3$ dan $g(n) = 100n^2 + 10n + 25$. Akan ditunjukkan $g(n) = O(f(n))$, akan tetapi $f(n) \neq O(g(n))$.

$$\lim_{n \rightarrow \infty} \frac{g(n)}{f(n)} = \lim_{n \rightarrow \infty} \frac{100n^2 + 10n + 25}{n^3}$$

$$\lim_{n \rightarrow \infty} \frac{g(n)}{f(n)} = \lim_{n \rightarrow \infty} \frac{200n + 10}{3n^2}$$

$$\lim_{n \rightarrow \infty} \frac{g(n)}{f(n)} = \lim_{n \rightarrow \infty} \frac{200}{6n} = 0$$

Kita lihat, karena nilai limitnya ada yaitu 0, maka $g(n) = O(f(n))$.

Sekarang kita lihat:

$$\lim_{n \rightarrow \infty} \frac{g(n)}{f(n)} = \lim_{n \rightarrow \infty} \frac{n^3}{100n^2 + 10n + 25}$$

$$\lim_{n \rightarrow \infty} \frac{g(n)}{f(n)} = \lim_{n \rightarrow \infty} \frac{3n^2}{200n + 10}$$

$$\lim_{n \rightarrow \infty} \frac{g(n)}{f(n)} = \lim_{n \rightarrow \infty} \frac{6n}{200} = \infty$$

Karena nilai limitnya tidak ada atau ∞ , maka $f(n) \neq O(g(n))$.

Contoh 3:

Algoritma Bubble Sort mempunyai kerja terburuk $\frac{n(n-1)}{2}$. Tunjukkan bahwa $\frac{n(n-1)}{2} = O(n^2)$.

Kalau kita lihat definisi order, $g(n) = O(f(n))$ bila $f(n)$ mendominasi $g(n)$ dan $g(n)$ didominasi oleh $f(n)$. Sehingga kita bisa menunjukkannya dengan cara:

$$\lim_{n \rightarrow \infty} \frac{g(n)}{f(n)} = K \text{ untuk } K \in R^+.$$

$K \in R^+$ berarti bahwa K tidak boleh bernilai 0.

Pada contoh ini, $g(n) = \frac{n(n-1)}{2}$ dan $f(n) = n^2$. Sekarang kita lihat nilai limit berikut ini.

$$\lim_{n \rightarrow \infty} \frac{g(n)}{f(n)} = \lim_{n \rightarrow \infty} \frac{n(n-1)/2}{n^2}$$

$$\lim_{n \rightarrow \infty} \frac{g(n)}{f(n)} = \lim_{n \rightarrow \infty} \frac{n - \frac{1}{2}}{2n}$$

$$\lim_{n \rightarrow \infty} \frac{g(n)}{f(n)} = \lim_{n \rightarrow \infty} \frac{1}{2} = \frac{1}{2}$$

Dengan demikian $\frac{n(n-1)}{2} = O(n^2)$.

Dengan demikian, untuk mengetahui apakah suatu fungsi berada dalam satu kelas dengan fungsi lainnya, kita dapat memakai order O. Atau O dapat kita pakai untuk menentukan bahwa suatu algoritma mempunyai kompleksitas sama dengan algoritma lainnya. Misal algoritma A mempunyai kompleksitas $W_1(n)$ dan algoritma B mempunyai kompleksitas $W_2(n)$, jika $W_1(n) = O(f(n))$ dan $W_2(n) = O(f(n))$ maka algoritma A dan B berada dalam satu kelas kompleksitas.

6. Lebih Jauh Tentang Order.

Kita akan melihat perbedaan kecepatan algoritma yang berbeda order (kelas) dalam menyelesaikan permasalahan. Pada Tabel 1.2 menampilkan waktu eksekusi dari beberapa algoritma yang berbeda dalam banyaknya kerja apabila diberikan input sebesar n . Dan menampilkan banyaknya ukuran input yang mampu diselesaikan dalam waktu 1 detik dan 1 menit. Pembaca dapat membedakan sendiri laju pertumbuhan beberapa fungsi. Perlu diingat bahwa faktor konstanta yang besar yang berada pada kelas algoritma $O(n)$ dan $O(n \log n)$ tidak mengakibatkan algoritma tersebut lebih lambat dibandingkan kelas algoritma $O(n^2)$, $O(n^3)$, dan $O(2^n)$ yang mempunyai faktor konstanta kecil.

Tabel 1.2
Laju Pertumbuhan Beberapa Fungsi

		Banyaknya Kerja Algoritma Untuk Input Berukuran n				
		$33n$	$46n \log n$	$13n^2$	$2.4n^3$	2^n
Waktu Eksekusi, Bila Ukuran n=	10	.00033 dtk	.0015 dtk	.0013 dtk	.0034 dtk	.001 dtk
	100	.003 dtk	.03 dtk	.13 dtk	3.4 dtk	4×10^4 abad
	1,000	.033 dtk	.45 dtk	13 dtk	.94 jam	
	10,000	.33 dtk	6.1 dtk	22 mnt	39 hari	-
	100,000	3.3 dtk	1.3 mnt	1.5 hari	108 tahun	-
Maks Ukuran Input, Dalam Waktu=	1 detik	30,000	2,000	280	67	20
	1 menit	1,800,000	82,000	2,200	260	26

Sekarang kita lihat dari sisi lain, dengan melihat perkembangan prosesor yang sangat pesat, kita akan meningkatkan kecepatan proses penyelesaian permasalahan melalui peningkatan kecepatan prosesor. Apakah peningkatan kecepatan prosesor ini banyak membantu ?. Untuk

menjawab ini akan kita lihat memlaui contoh. Misal ada dua orang Pak Untung dan Pak Harto. Keduanya ingin melakukan pengurutan data sebanyak 1.000.000 data. Pak Untung memakai komputer PC lama yang mempunyai kecepatan proses 10 MIPS (Mega Instruction Per Second), dan dalam memprogram pengurutan menggunakan algoritma *merge sort* dengan kerja $50n \log n$. Pak Harto memakai komputer main frame yang mempunyai kecepatan proses 200 MIPS (20 kali lebih cepat dari komputer yang dipakai Pak Untung), dan dalam memprogram pengurutan menggunakan algoritma *insertion sort* dengan kerja $2n^2$. Akan kita lihat waktu yang dibutuhkan dari keduanya.

Waktu untuk hasil kerja Pak Harto:

$$\frac{2(10^6)^2 \text{ instruksi}}{2(10^8) \text{ instruksi/detik}} = 10.000 \text{ detik} = 2,78 \text{ jam}$$

Waktu untuk hasil kerja Pak Untung:

$$\frac{50(10^6) \log 10^6 \text{ instruksi}}{10^6 \text{ instruksi/detik}} = 996,6 \text{ detik} = 16,6 \text{ menit}$$

Dari contoh di atas menunjukkan bahwa peningkatan kecepatan algoritma (pemilihan algoritma) jauh lebih baik dari pada meningkatkan kecepatan prosesor. Hal ini menunjukkan bahwa perangkat lunak (algoritma) merupakan suatu teknologi.

BAB II

PENCARIAN ARRAY TERURUT

Pandang permasalahan pencarian elemen x pada suatu array A dengan n elemen terurut naik atau $A[1] < A[2] < \dots < A[n]$. Kita ingin mendapatkan index dari x pada array A , bila $x = A[i]$ maka hasilnya adalah i . Namun apabila x tidak ada di dalam array A hasilnya adalah -1.

Untuk menyelesaikan permasalahan ini, jelas algoritma *SequentialSearch* yang ada di depan dapat kita pakai. Kalau kita memakai algoritma *SequentialSearch* berarti tidak ada bedanya antara array yang sudah terurut maupun belum terurut. Mestinya dengan keadaan array yang terurut ini, kita dapat membuat algoritma yang memanfaatkan keterurutan elemen array ini. Barang kali algoritmanya lebih baik dari pada *SequentialSearch*.

Dalam perbandingan dengan x , kita langsung menuju elemen tengah (katakan pada index ke k) dan kita bandingkan $A[k]$ dengan x . Jika x sama dengan $A[k]$ maka keluarkan hasil k . Namun apabila x tidak sama dengan $A[k]$, kemungkinannya adalah x lebih kecil dari $A[k]$ atau lebih besar dari $A[k]$. Jika x lebih kecil dari $A[k]$, maka pasti x tidak sama dengan $A[j]$ untuk $j > k$. Untuk itu kita cari x pada $A[k+1]$ sampai dengan $A[n-1]$. Kemungkinan lainnya, yaitu apabila x lebih besar dari $A[k]$ maka x tidak sama dengan $A[i]$ untuk $i < k$. Untuk itu kita cari x pada $A[0]$ sampai dengan $A[k-1]$. Langkah ini kita ulang terus hingga kita dapatkan hasil. Langkah - langkah ini dikenal dengan sebutan algoritma *binary Search* dan dapat kita tuliskan sebagai berikut.

Input : Array A dengan n elemen bilangan terurut naik dan bilangan x yang dicari.

Output : Bilangan bulat non negatif j yang merupakan letak / index x di dalam array A .

Menghasilkan $j = -1$ bila x tidak ada di A .

Algoritma:

BinarySearch(A, n, x) {

1. $k=0$; $m=n$;

2. *while* ($k < m$) {

3. $j = \left\lfloor \frac{k+m}{2} \right\rfloor$

4. *if* ($x = A[j]$) *return* j ;

5. *else if* ($x < A[j]$) $m=j-1$;

6. *else* $k=j+1$;

}

$j=-1$;

}

Analisis Banyak Kerja Terburuk